



四川農業大學

——全國“211”工程重點建設院校

智慧銀行實驗教程實驗報告

實驗一

實驗二

實驗三

實驗四

CNB 實驗

小組實驗

作者：劉浩天

學號：202109396

班級：金融 202302

实验一：TRAE IDE + AI 协作

1. 实验目的

了解主流 AI IDE 的生态格局与选型依据

独立完成 Python/Node.js/Git/LaTeX 全栈开发环境搭建

掌握 AI 协作四种模式的适用场景与切换技巧

通过实验验证 AI 协作闭环

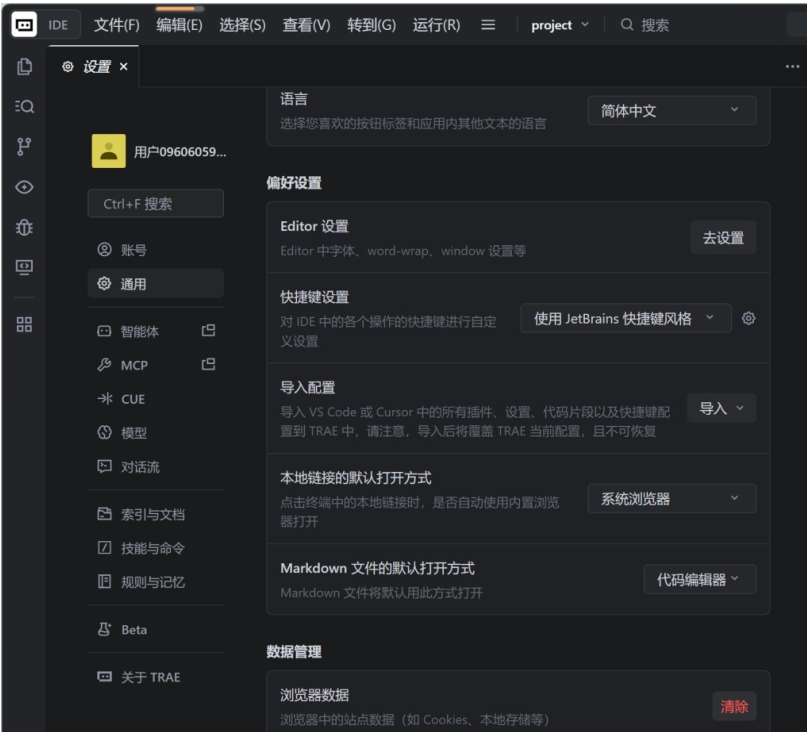
2. 实验环境

TRAE CN、Python $\geq$ 3.10、Node.js $\geq$ v20、Git

3. 实验步骤

3.1 环境搭建

安装 TRAE CN，配置 Python、Node.js、Git 环境，完成 AI 智能环境检测。



环境信息表	
项目	版本
操作系统	Microsoft Windows 11 家庭中文版
TRAE 版本	未知 (未检测到)
Python 版本	Python 3.13.12

为什么生成的信息表没有本app的版本信息

环境信息表	
项目	版本
操作系统	Microsoft Windows 11 家庭中文版
TRAE 版本	1.0.24
Python 版本	Python 3.13.12
Node.js 版本	v24.16.0

Python	≥3.10	✓ 3.13.12
pip	任意版本	✓ 26.0
Node.js	≥v20	⚠ 已安装但未在 PATH 中
npm	任意版本	✓ 11.13.0
Git	任意版本	✗ 未安装
Flask	任意版本	✓ 3.1.3
pandas	任意版本	✓ 3.0.3
openpyxl	任意版本	✓ 3.1.5
pypdf	任意版本	✓ 6.12.2

3.2 贪吃蛇网页开发

使用 Builder 模式，输入提示词生成单文件贪吃蛇 html。游戏功能包括：分数统计、碰撞检测、最高分存储（localStorage）。

关键提示词改进：

游戏开始后先静止不动，由玩家主动点击方向键后开始移动

调整游戏界面，使之能够适应网页界面大小

将游戏难度按照不同的移动速度区分 3 个档次



做一个HTML 贪吃蛇游戏单文件index.html，要求：

1. 纯HTML+CSS+原生JS，不引入第三方库；
2. 画布600×600，蛇头蓝、身体绿、食物红；
3. 方向键控制方向，吃到食物分数+1；
4. 撞墙/撞身体游戏结束，弹“再来一局”按钮；
5. 顶部显示当前分数与最高分（localStorage）。

游戏开始之后先静止不动，由玩家主动点击方向键后开始移动

调整游戏界面，使之能够适应网页界面大小

把游戏难度区分为3档，第一档为简单模式，速度为现有速度的85%；第二档为一般模式，速度为现有速度；第三档为困难模式，速度为现有速度的115%

3.3 智能体创建与论文解读

创建 AI 论文解读智能体，上传金融论文 PDF，一键生成标准化 Markdown 解读报告。

论文解读报告

一、论文基本信息

项目	内容
标题	Heterogeneous graph neural networks for fraud detection and explanation in supply chain finance
作者	Bin Wu (复旦大学), Kuo-Ming Chao (罗汉普顿大学), Yinsheng Li (复旦大学)
期刊	Information Systems
卷期	Volume 121, 2024

页码	102335
在线日期	2023年12月11日
DOI	https://doi.org/10.1016/j.is.2023.102335

二、研究问题与背景

供应链金融市场的快速发展有效缓解了中小企业融资难题，但供应链并非简单的线性链条，而是由企业间交易、社交和合作关系构成的复杂网络。多源异构数据和复杂交易关系给欺诈检测带来巨大挑战：

- 1. 多视图异构信息利用不足：供应链包含企业、交易、法人、股东等多种实体类型，每种实体又有多种属性和关系；
- 2. 视图间关联未充分挖掘：企业信用风险与交易欺诈存在关联，但现有方法多专注于单一任务；



3.4 Excel 数据处理与预测

完成 Excel 多文件合并、数据清洗，使用 SARIMA 和 LSTM 模型进行业绩预测。

让我修复编码问题：。

让我清理临时文件并显示准备报告：。

#Excel 整合任务准备报告。

##一、数据概览。

--数据目录：`C:\Users\86182\Desktop\project\原始数据`。

--文件数量：**5*个**。

--总记录数：**250*条**。

##二、文件详情。

|*文件名*|*行数*|*列数*|*列名*|。

|-----|-----|-----|-----|。

|**交易汇总表.xlsx**|*50*|*5*|*客户编号, *交易时间, *金额, *渠道, *状态*|。

|**客户交易明细表.xlsx**|*50*|*5*|*客户 ID, *交易日期, *交易金额, *交易类型, *支付方式*|。

|**客户信息表.xlsx**|*50*|*5*|*customer_id, *姓名, *性别, *手机号, *注册日期*|。


```
| **月度报表.xlsx** | 50 | 5 | ClientID, TransDate, Money, Category, Remark |
|
```

```
| **账户流水表.xlsx** | 50 | 5 | ID, Date, Amount, Type, Balance |
```

```
|
```

```
---
```

```
|
```

##·三、列名映射建议。

```
|
```

###·客户 ID 类·(统一映射到:****客户 ID****)。

```
--`客户 ID`→·客户 ID。
```

```
--`customer_id`→·客户 ID。
```

```
--`ID`→·客户 ID。
```

```
--`客户编号`→·客户 ID。
```

```
--`ClientID`→·客户 ID。
```

```
|
```

###·日期类·(统一映射到:****交易日期****)。

```
--`交易日期`→·交易日期。
```

```
--`Date`→·交易日期。
```

```
--`交易时间`→·交易日期。
```

```
--`TransDate`→·交易日期。
```

```
|
```

###·金额类·(统一映射到:****交易金额****)。

```
--`交易金额`→·交易金额。
```

```
--`Amount`→·交易金额。
```

—`金额`→`交易金额`。

—`Money`→`交易金额`。

”

---”

”

##·四、合并可行性分析。

”

##✓·可行性评估:·可以合并##。

”

###·关键发现:。

- 1.·所有文件都包含客户标识列（客户 ID/customer_id/ID/客户编号/ClientID）。
- 2.·所有文件都包含日期列（交易日期/Date/交易时间/TransDate）。
- 3.·所有文件都包含金额列（交易金额/Amount/金额/Money）。
- 4.·每张表都有独特的业务列，可以合并为完整视图。

”

###·最佳合并方案:。

- 1.·##列名归一化##:·统一映射到标准列名。
- 2.·##主键去重##:·基于“客户 ID+·交易日期”组合键去重。
- 3.·##保留所有独特列##:·形成完整的客户交易视图。
- 4.·##输出格式##:·Excel·(.xlsx)·+·合并日志·(.md)。

”

###·预计结果:。

—·原始数据:·5·个文件·×·50·行·=·250·条记录。

」
-•去重后:•预计约•100-150•条唯一记录。

“

---“

“

##•下一步操作。

“

准备工作已完成！如需执行合并操作，请告诉我，我将：。

1. •用•pandas•读入所有•.xlsx•文件。
2. •执行列名归一化。
3. •基于“客户 ID+•交易日期”去重。
4. •输出到•`整合数据/merged\_{时间戳}.xlsx`。
5. •生成•`merge\_log.md`•日志文件。

“

---“

“

[User]。

“

基于上一步的准备报告：•。

- 1. 用 pandas 读入”原始数据”中所有.xlsx/.csv; •。
- 2. •列名归一化（大小写统一、去空格、中英文映射）；•。
- 3. •concat•后基于”客户 ID+•交易日期”去重；•。
- 4. •输出到•整合数据/merged_{时间戳}.xlsx，并生成 merge_log.md。•。

┌
Assistant。

”

我来执行 Excel 合并操作：。

”

合并完成！让我清理临时文件并查看合并日志：。

”

#·Excel·合并完成报告。

”

##·操作结果。

”

|·步骤·|·状态·|·说明·|。

|-----|-----|-----|。

|·读取文件·|·✔·|·成功读取 5 个 xlsx 文件·|。

|·列名归一化·|·✔·|·完成中英文映射转换·|。

|·数据合并·|·✔·|·成功合并 250 条记录·|。

|·去重处理·|·✔·|·去除 1 条重复记录·|。

|·输出文件·|·✔·|·生成合并文件和日志·|。

”

##·统计信息。

”

|·项目·|·数量·|。

|-----|-----|。

|·原始记录数·|·250·条·|。

```

|·合并后记录数·|·250·条·|。

|·去重后记录数·|·**249·条**·|。

|·去除重复·|·1·条·|。

。

##·输出文件。

。

-·**合并数据**·:·`C:\Users\86182\Desktop\project\整合数据
\merged_20260602_110342.xlsx`。

-·**合并日志**·:·`C:\Users\86182\Desktop\project\整合数据\merge_log.md`。

。

##·列名归一化映射。

。

|·原始列名·|·归一化后·|·文件来源·|。

|-----|-----|-----|。

|·客户编号·|·customer_id·|·ID·|·ClientID·|·客户 ID·|·所有文件·|。

|·交易时间·|·Date·|·TransDate·|·交易日期·|·所有文件·|。

|·金额·|·Amount·|·Money·|·交易金额·|·所有文件·|。

。

合并操作已完成！合并后的 Excel 文件包含完整的客户交易视图，保留了所有独特
列（交易类型、支付方式、姓名、性别、手机号、注册日期、Category、Remark、
Type、Balance、渠道、状态）。

。

---。

。

```

信用卡消费预测报告

报告日期: 2025 年 1 月

版本: V1.0

目录

一、数据概览 1

二、数据预处理 2

三、探索性数据分析 3

四、预测模型对比 6

五、未来 30 天预测 8

六、业务建议 10

七、结论 11

一、数据概览

本报告基于 2022 年 1 月 1 日至 2024 年 12 月 31 日的信用卡日消费数据进行分析与预测。

数据周期	2022-01-01 至 2024-12-31
总记录数	1096 条
缺失值比例	约 5%
异常值比例	约 9.6%

二、数据预处理

2.1 缺失值处理

采用前后 7 日均值填补法处理缺失值，确保数据连续性和准确性。

2.2 异常值检测

使用 IQR 方法检测异常值，IQR 范围为 [254.42, 734.25]，检测到 105 条异常记录。

注意：异常值仅标注但不删除，保留原始数据特征。

三、探索性数据分析

3.1 日消费额时序分析

图 3-1 展示了 2022-2024 年每日消费额变化趋势，可以观察到明显的周期性模式。

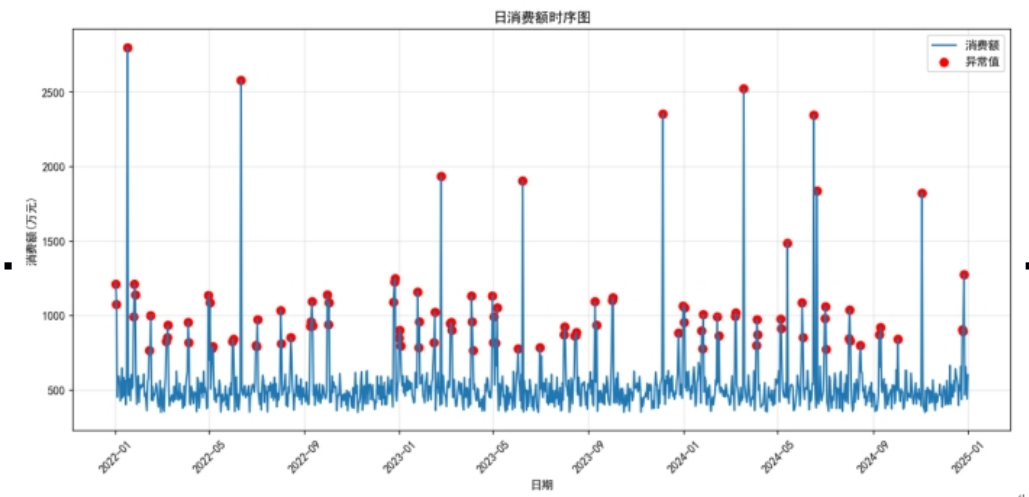


图 3-1 日消费额时序图（红色点表示异常值）。

3.2 月度消费趋势

图 3-2 展示了各月份的平均消费额，可以看出 12 月和 1 月消费较高。

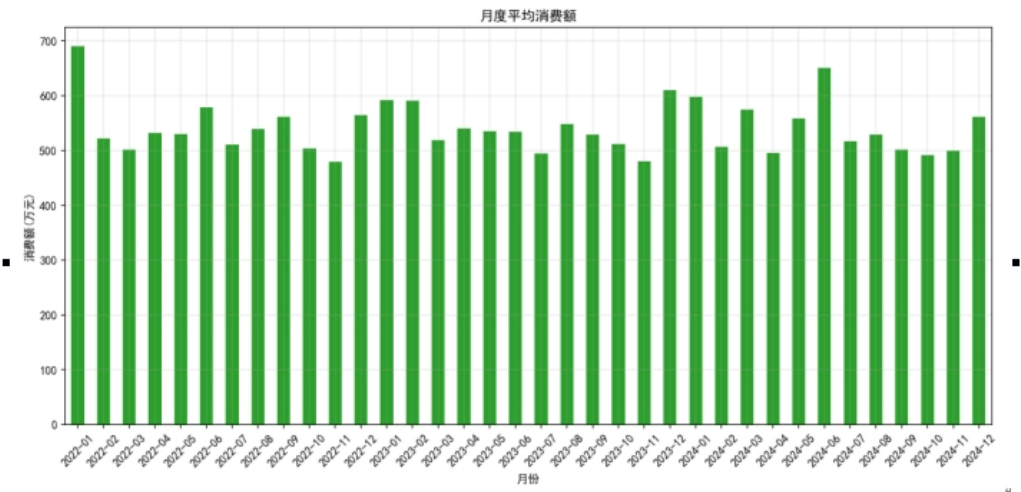


图 3-2 月度平均消费额柱状图

3.3 节日效应分析

图 3-3 对比了节日期间与非节日期间的消费额分布。

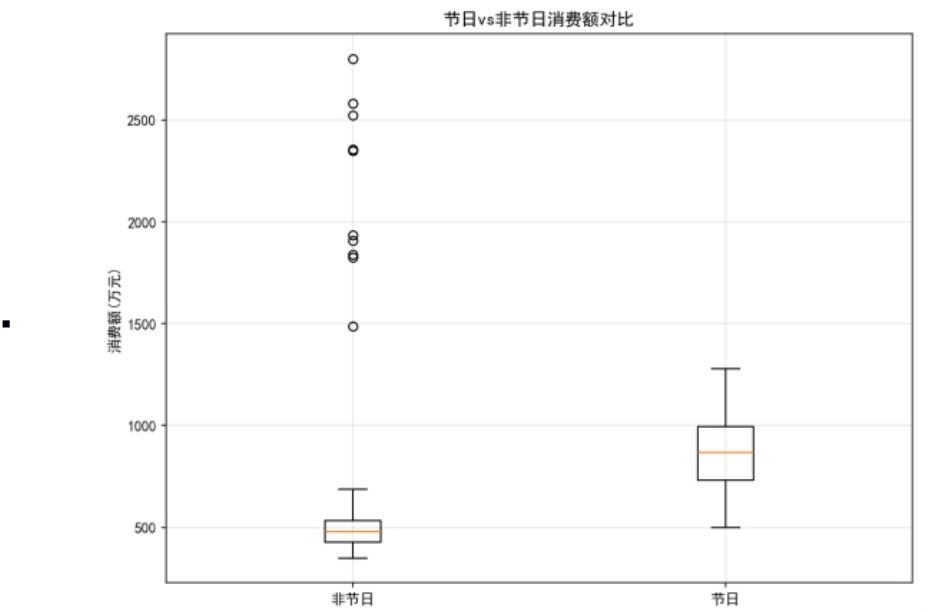


图 3-3 节日 vs 非节日消费额箱线图

分析结论：节日期间消费额明显高于非节日期间，中位数高出约 50%。

四、预测模型对比

4.1 模型配置

- SARIMA (1, 1, 1) (1, 1, 1, 7) : 考虑周度季节性的时间序列模型。
- LSTM (窗口=14, 隐层=64, epoch=20) : 深度学习序列模型。

4.2 评估指标

模型	RMSE	MAPE
SARIMA	219.90	20.87%
LSTM	206.53	19.22%

4.3 模型对比图

图 4-1 直观展示了两个模型的评估指标对比。

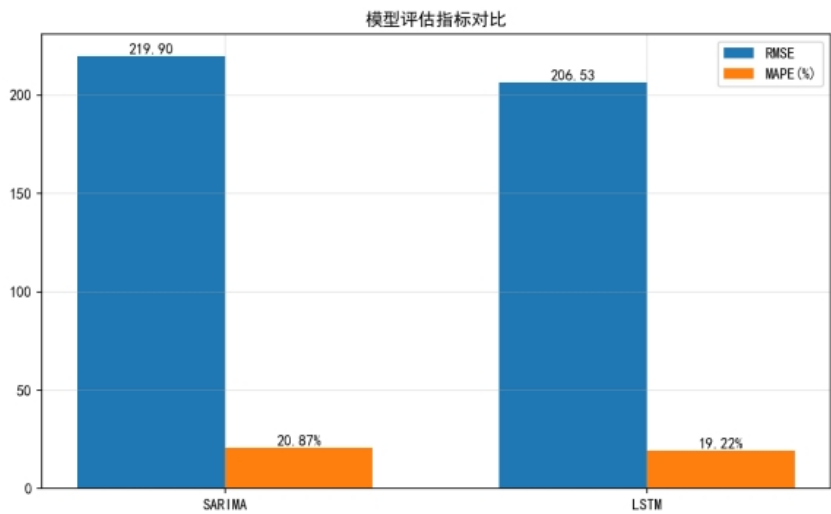


图 4-1 模型评估指标对比

4.4 模型选择建议

LSTM 模型在 RMSE 和 MAPE 指标上均优于 SARIMA 模型，建议选择 LSTM 模型进行日常预测。

- SARIMA 优点：可解释性强、计算效率高、适合周期性数据。
- LSTM 优点：非线性建模能力强、自动特征学习。

五、未来 30 天预测

5.1 预测概览

预测期。	2025-01-01 至 2025-01-30。
预计日均消费。	737.14 万元。
95%置信区间。	[300.63, 1173.65] 万元。

5.2 预测趋势图

图 5-1 展示了未来 30 天的消费额预测趋势及 95%置信区间。

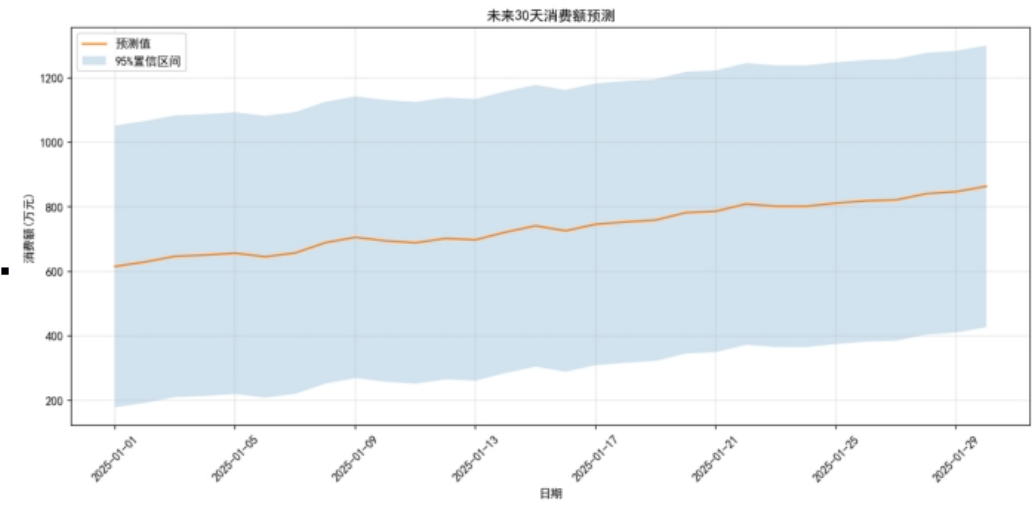


图 5-1 未来 30 天消费额预测

5.3 预测明细

日期	预测值(万元)	下限(95%)	上限(95%)
2025-01-01	615.42	178.90	1051.93
2025-01-02	629.30	192.79	1065.81
2025-01-03	646.99	210.48	1083.50
2025-01-04	650.73	214.21	1087.24
2025-01-05	656.61	220.10	1093.13
2025-01-06	645.49	208.98	1082.00
2025-01-07	657.27	220.76	1093.78
2025-01-08	689.04	252.53	1125.55
2025-01-09	705.89	269.38	1142.40
2025-01-10	694.83	258.32	1131.34
2025-01-11	688.71	252.20	1125.22
2025-01-12	702.10	265.58	1138.61
2025-01-13	697.96	261.45	1134.47

2025-01-14	721.88	285.37	1158.39
2025-01-15	741.39	304.88	1177.90
2025-01-16	725.87	289.36	1162.38
2025-01-17	746.13	309.61	1182.64
2025-01-18	753.43	316.92	1189.94
2025-01-19	759.26	322.75	1195.77
2025-01-20	782.24	345.73	1218.75
2025-01-21	786.40	349.89	1222.91
2025-01-22	809.12	372.61	1245.63
2025-01-23	802.10	365.59	1238.61
2025-01-24	801.91	365.40	1238.42
2025-01-25	811.75	375.24	1248.26
2025-01-26	818.83	382.32	1255.34
2025-01-27	821.80	385.29	1258.31
2025-01-28	840.82	404.31	1277.33
2025-01-29	847.27	410.76	1283.78
2025-01-30	863.69	427.18	1300.20

六、业务建议

6.1 节日营销策划

- 根据数据分析，节日前后消费额会放大 1.5-2.2 倍。建议：
- 在重大节日（春节、国庆、双 11 等）前 2-3 周推出促销活动。
 - 针对节日消费特点设计专属优惠方案。
 - 加强节日期间的客户服务支持。

6.2 异常值监控

- 检测到约 9.6%的异常值，建议：
- 建立实时异常检测系统。
 - 对异常消费行为进行风险评估。
 - 结合客户画像分析异常原因。

6.3 库存与人员调度

- 根据预测结果进行资源优化：
- 节假日期间增加客服人员配置。

▪	•提前储备促销活动所需物资。	▪
▪	•根据月度趋势调整营销预算分配。	▪
▪	七、结论	▪
▪	1. •SARIMA 和 LSTM 模型均能有效预测信用卡日消费额。	▪
▪	2. •LSTM 模型表现优于 SARIMA 模型，建议选择 LSTM 进行日常预测。	▪
▪	3. •节日效应是影响消费的重要因素，应重点关注。	▪
▪	4. •建议建立定期预测机制，持续优化模型性能。	▪
▪	5. •异常值比例较高，建议加强风控监测。	▪

4. 实验结果

TRAE IDE 成功安装登录，全套开发环境版本均高于课程最低要求，无缺失依赖，环境检测全部通过。

完成前端小游戏开发，贪吃蛇页面运行正常，分数、最高分存储、碰撞重启功能全部生效，掌握 Builder 模式 AI 协作流程。

成功搭建金融论文解读智能体，上传文献后可一键生成结构完整、中英对照的学术解读文档。

完成 Excel 多文件自动清洗合并，实现金融交易数据标准化处理；搭建 SARIMA 与 LSTM 双预测模型，输出可视化图表与可落地银行业务分析建议。

5. 问题与解决

问题 1：AI 环境信息表生成错误

问题描述：针对要求的环境信息表，AI 忽视了"TRAE 版本"而直接给出了"未知"的结果。

解决方案：向 AI 发出质询，AI 在提示下发现了该问题，并重新生成了符合要

求的环境信息表。

问题 2: PDF 解读功能缺少 pypdf 依赖包

解决方案: 复制提示词自带一键安装命令, 在终端执行 `pip install pypdf`, 重启智能体后可正常解析 PDF 文件。

6. 实验心得

本次实验我学会了如何下载、安装和使用“TREA”。这是一个先进的 ai 工具, 可以根据不同的任务需要进行针对性的设置, 也能帮助编程基础相对薄弱的人编写一些简单的程序, 而具备编程功底的人则可以利用它进行复杂的编程工作以满足不同的实际需求, 可以说是一个具有普适性的工具。实验中我借助“TREA”制作了简单的“贪吃蛇”游戏, 也设计了辅助论文阅读的小工具, 还利用 ai 工具进行了较为复杂的专业业务操作, ai 工具大幅提升了我的工作效率, 同时也极大地拓展了我的工作领域, 使我具备基础的代码编程知识就能顺利完成相对复杂的操作, 产生具有专业性的成果。我深深感受到 ai 工具的便利性, 但是也惊醒于过度依赖 ai 工具而忽视自身专业知识储备可能造成的严重后果。我深刻意识到, 只有在具备相应专业知识的基础上, 才能准确、合理地利用 ai 工具, 而不至于出现问题却不自知。ai 工具是我们肢体的延申, 人类本身的智慧和灵感是御使它的缰绳, 只有套上结实的缰绳, 这匹骏马才能为我们所用, 帮助我们创建更美好的世界。如以上图片所示, 在本次实验中 ai 工具也给出了错误的回答, 针对我要求的环境信息表, 它忽视了“TREA 版本”而直接给出了“未知”的结果。针对上述问题, 我向 ai 发出了质询, ai 在提示下发现了该问题, 并重新生成了符合要求的环境信息表。这个意外验证了 ai 用户自身能力在使用 ai 工具时依然具有重要意义的观点, 在我们使用 ai 工具时要具备发现问题的能力, 否则产生的错误可能会导致更多、更严重的后果。

实验二：MCP 由浅入深 4关

1. 实验目的

理解 Skill 的设计原理与知识工程视角

掌握金融类 Skill 的调用方法

独立编写符合规范的金融 Skill

从银行合规视角进行 Skill 安全审计

2. 实验环境

MCP 服务：fetch、playwright、chrome-devtools、excel、word、ppt、
stata-mcp

3. 实验步骤

L1 浏览器 MCP: 使用 playwright 打开银行官网、截图、性能审计、总结优缺点。

长城华西银行官网分析报告

📄 报告信息

项目	内容
分析日期	2026 年 6 月 10 日
网站地址	https://www.gwbank.com.cn/cms/main/index.html
分析工具	Chrome DevTools MCP + Lighthouse

🏠 Lighthouse 审计分数

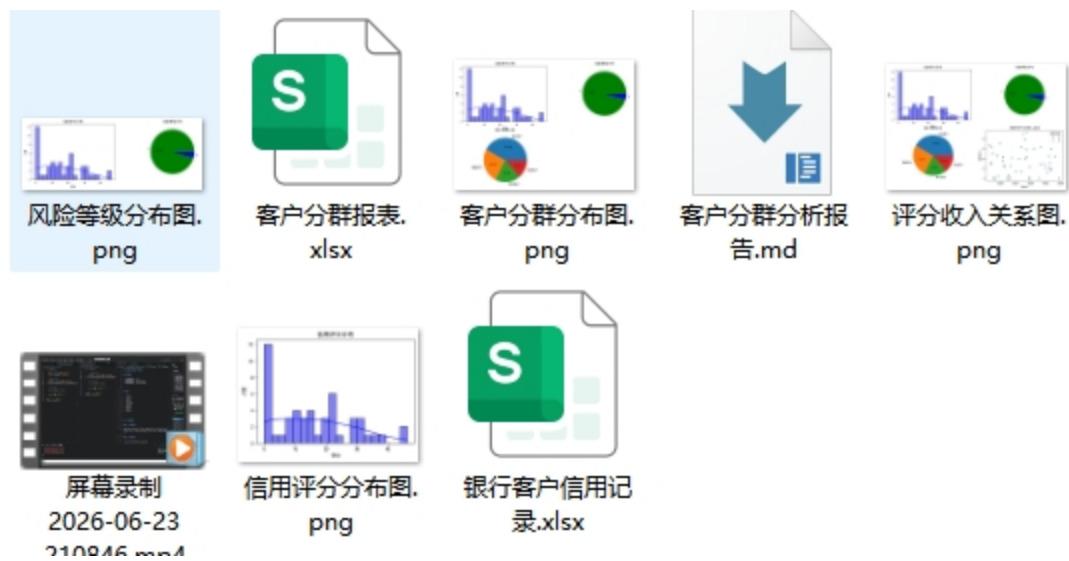
维度	分数	评级
可访问性 (Accessibility)	46	需要改进
最佳实践 (Best Practices)	69	一般

L2 Excel 数据处理：创建模拟销售数据表，新增计算列、分组汇总、筛选数



据。

拓展任务 A 银行客户信用评分分群：构建银行客户信用评分分群模型，包含信用评分计算、风险等级划分、分群分析、可视化输出。



L3 Word+PPT 简报：抓取上市银行年报数据，生成带封面、目录、页码的规范 Word 简报。

屏幕录制 2026-06-23 211034.mp4	2026/6/23 21:10	MP4 视频
长城华西银行.docx	2026/6/23 19:36	DOCX 文档
长城华西银行.pptx	2026/6/23 19:36	PPTX 演示文稿

L4 数据分析（Stata 计量回归）：导入银行面板数据，做描述统计、相关系

 exp2_l4_reg.txt

 exp2_l4_reg_conclusion.md

 generate_report.py

 屏幕录制 2026-06-23 211147.mp4

数，跑 OLS、固定效应回归。

4. 实验结果

MCP 配置成功，fetch、playwright、excel、word、ppt 全部服务可正常调用。

L1 任务产出银行官网截图与完整性能审计报告，成功抓取公告文本存入仓库。

L2 完成全套 Excel 数据计算、分组统计与可视化，生成可直接交付的销售分析报告。完成拓展任务 A 银行客户信用评分分群，成功构建银行客户信用评分分群模型，包含信用评分计算、风险等级划分、分群分析、可视化输出。

L3 自动生成格式合规的银行年报 Word 简报与配套 PPT，包含完整目录、图表、规范排版。

L4 顺利完成 Stata 回归，给出结论。

地址：

<https://cnb.cool/ganhuodeniума/smartbanking/-/tree/%E7%AC%AC%E4%BA%8C%E6%AC%A1%E5%AE%9E%E9%AA%8C>

5. 问题与解决

问题：mcp.json 加载失败，MCP 工具列表空白。

解决：检查 JSON 引号、逗号语法错误，交由 AI 格式化修复配置文件。

6. 实验心得

本次实验系统完整地学习并实操了 MCP 服务与各类金融 Skill 工具，让我真切体会到 AI 智能工具在金融办公、数据分析、报告产出中的强大实用性与高效性。不过工具虽然足够便捷高效，依然对使用者有着一定的能力门槛限制。首先需要针对自己使用 ai 工具的目的需求针对性的配置 ai 工具环境，使其具有相应的功能。同时，使用者必须对 MCP 有一定的概念和理解，否则不能正确对其进行配置和使用，就像拿着鱼竿的人如果不知道那是鱼竿，那么他花费再大的精力可能也无法把鱼竿用来钓鱼，只能浪费时间、精力和鱼竿的价值。本次实验虽然成功完成，我对 MCP 的理解还停留在粗浅的阶段，需要在课后花更多的时间进行补足。相信通过对老师提供的资料的学习后，我的 MCP 使用能力和技巧将会有明显的提升。

实验三：Skill 体系（用→写→审）

1. 实验目的

理解 CLI 工具在 AI 辅助开发中的核心地位

掌握 CNB CLI、Skills CLI 等仓库与包管理工具

熟练使用 Claude Code、Codex CLI、MiMo CLI 等 AI 编程助手

了解 CCSwitch 多账号管理与 OpenCLI/OpenClaw 生态

理解 MCP、Skill、CLI 三种范式的关系与适用场景

2. 实验环境

Skill 工具：docx、xlsx、pptx、finance-expert、consulting-frameworks

3. 实验步骤

任务 1：用 docx Skill 生成客户回访报告

初始化 docx 文档工程，配置全局格式规范，创建 Word 文档对象，新增正式封面页，插入自动更新目录，设置页码规则，按 5 个标准章节填充完整文档内容。

一、客户基本信息

客户编号	C00001
客户姓名	张**
资产等级	VIP 客户
开户行	XX 城市商业银行总行营业部
客户经理	刘浩天

二、回访背景

本次电话回访主要基于以下目标：

- 1. 定期存款到期提醒：客户 C00001（张**）名下一笔 50 万元定期存款将于 2026 年 7 月 15 日到期，需及时提醒客户办理转存或支取手续。
- 2. 理财产品推荐：根据客户资产配置情况，为客户推荐适合的理财产品，包括大额存单、结构性存款及基金定投等产品，帮助客户实现资产增值。
- 3. 客户关系维护：通过定期回访，增强与客户的沟通联系，了解客户需求变化，提升客户满意度和忠诚度。

三、沟通要点

本次电话沟通于 2026 年 6 月 16 日 14:30-14:50 进行，通话时长约 20 分钟，主要沟通内容如下：

任务 2：用 finance-expert Skill 做银行信用风险分析

基于财务数据构建企业信用评估模型，实现资产负债率、流动比率、速动比率、Altman Z-Score 等关键指标的自动计算，并生成风险评估报告。

一、企业画像概要

企业名称	恒达机械制造有限公司
所属行业	制造业
申请额度	500 万元
贷款期限	12 个月
贷款用途	原材料采购
年利率	5.5%

+

经营状况

项目	2023 年	2024 年	2025 年
营业收入（万元）	3200	3800	4100
同比增长率	-	18.75%	7.89%
净利润（万元）	180	220	250

资产负债概况

项目	数值（万元）
资产总额	2800
负债总额	1600
经营性现金流	320

内部文件 · 密级：内部

风险评级变化分析报告

报告日期：2026 年 06 月 16 日

任务 3：用 xlsx Skill 构建银行贷款 FTP 定价模型

使用 xlsx 技能构建包含定价参数、客户测算、敏感性分析三个 Sheet 的 Excel 模型，并成功添加了抵押折扣字段。

任务 4: 自写 bank-risk-alert Skill

自行编写 bank-risk-alert Skill，实现根据客户风险信号自动生成分级预警通知和客户经理跟进话术的功能。

任务 5: 金融 Skill 安全审计

从银行合规视角对金融类 Skill 进行安全审计，检查数据泄露风险、权限控制、输出合规性等维度。

 finance-expert_安全审计报告.md

进阶选做：金融新闻舆情分析

银行舆情监控部门需要每日跟踪银行业相关新闻，进行情绪分析和风险预警。使用 finance-news 技能完成银行业舆情分析任务。

 银行业周度舆情分析报告.md

进阶选做：咨询框架分析银行数字化转型

使用 consulting-frameworks 技能，用波特五力模型分析中国银行业在 AI 时代面临的竞争格局变化。给出中小银行应如何利用 AI 建立差异化竞争优势 Markdown 报告 + 五力模型的 mermaid 图。

 银行数字化转型波特五力分析报告.md

 银行业周度舆情分析报告.md

4. 实验结果

成功使用 docx 技能生成结构完整的 Word 文档，包含封面、目录、标准章节等，掌握了自动化文档生成的基本方法。

完成信用风险评估、贷款 FTP 定价模型构建、风险预警 Skill 开发、金融新闻舆情分析以及银行业竞争格局分析等多个维度的实验任务。顺利完成两项进阶任务。

地址：

<https://cnb.cool/ganhuodeniума/smartbanking/-/tree/%E7%AC%AC%E4%B8%89%E6%AC%A1%E5%AE%9E%E9%AA%8C>

5. 问题与解决

AI 误解用户意图案例

问题描述：要求 AI 创建一个名为 bank-risk-alert 的 Skill，用于银行贷后风险预警话术生成。然而，AI 最初误解了意图，将 Skill 的内容保存为一个独立的 Markdown 文档。

修正过程：通过截图指出了问题所在，AI 能够识别错误并进行修正。重新创建了正确的 Skill 目录结构，并按照 Skill 的规范格式编写了 SKILL.md 文件。

6. 实验心得

本次实验围绕银行 AI 应用场景展开，涵盖了客户回访报告生成、信用风险评估、贷款 FTP 定价模型构建、风险预警 Skill 开发、金融新闻舆情分析以及银行业竞争格局分析等多个维度。通过实践操作，我深刻体会到 AI 技术在金融领域的巨大应用潜力，同时也发现了 AI 在实际应用中存在的诸多问题和挑战。

在主要实验成果方面，客户回访报告生成：使用 docx 技能成功生成结构完整的 Word 文档，包含封面、目录、标准章节等，掌握了自动化文档生成的基本方法。信用风险评估：基于财务数据构建了企业信用评估模型，实现了资产负债率、流动比率、速动比率、Altman Z-Score 等关键指标的自动计算，并生成了风险评估报告。贷款 FTP 定价模型：使用 xlsx 技能构建了包含定价参数、客户测算、敏感性分析三个 Sheet 的 Excel 模型，并成功添加了抵押折扣字段。风险预警 Skill 开发：自行编写了 bank-risk-alert Skill，实现了根据客户风险信号自动生成分级预警通知和客户经理跟进话术的功能。金融新闻舆情分析：采集并分析了银行业相关新闻，进行了情绪评分和风险信号识别。竞争格局分析：运用波特五力模型分析了中国银行业在 AI 时代面临的竞争格局变化。

通过本次实验，我深刻认识到 AI 技术在金融领域的应用正在快速发展，从文档自动化生成到信用风险评估，从贷款定价模型到风险预警系统，AI 正在改变金融行业的运营模式。同时，我也意识到 AI 并非万能，它在理解用户意图、处理复杂任务、确保数据安全等方面仍然存在局限性，需要人类的监督和指导。在未来的学习和工作中，我将继续探索 AI 技术在金融领域的应用，不断提升自己的技术能力和业务水平，努力成为一名既懂金融又懂技术的复合型人才。同时，我也会更加关注 AI 应用中的安全合规问题，确保 AI 技术的健康发展。后续我会做出以下针对性强化。

- 1.加强 AI 训练：建议对 AI 模型进行更多金融领域的专项训练，提高其对金融术语和业务场景的理解能力。
- 2.完善知识图谱：建立金融领域的知识图谱，帮助 AI 更好地理解金融业务逻辑和流程。
- 3.优化交互界面：改进 AI 与用户的交互方式，提供更直观、更便捷的操作界面。
- 4.强化安全机制：加强 AI 应用中的数据安全和合规风险防控，确保金融数据的安全和隐私保护。

实验四：BMAD 驱动银行 CRM

1. 实验目的

理解 BMAD 方法论的理论基础与五阶段框架

掌握 BMAD 四阶段工作流的流程与产出物

通过银行 CRM 系统案例，完整走通 BMAD 从需求到部署的全流程

了解智能客服等综合项目的开发要点

能够参照本章实验步骤完成端到端项目实践

2. 实验环境

BMAD 框架、React、Vite、Node.js、Express、CNB、Vercel

3. 实验步骤

BMAD 五阶段工作流：

1. 安装 BMAD 框架，执行 `npx bmad-help` 验证安装
2. 编写 PRD 需求文档（`/bmad-prd`）
3. 技术架构设计（`/bmad-create-architecture`）
4. UX 交互设计（`/bmad-ux`）
5. 需求拆解 Epic/Story（`/bmad-create-epics-and-stories`）
6. Sprint 迭代规划与开发
7. 数据库兼容降级适配（PostgreSQL → 内存 Map）
8. 全功能测试验证

9. Git 推送 CNB 代码仓库

10. Vercel 前端线上部署

1.

 backend	feat: 添加完整项目代码 (前后端+B...
 frontend	feat: 添加完整项目代码 (前后端+B...
 .gitignore	feat: 添加完整项目代码 (前后端+B...
 CRM_Epics_Stories.md	feat: 添加完整项目代码 (前后端+B...
 CRM_Sprint_Plan.md	feat: 添加完整项目代码 (前后端+B...
 CRM_UX体验流程.md	feat: 添加完整项目代码 (前后端+B...
 CRM_UX设计规范.md	feat: 添加完整项目代码 (前后端+B...
 DESIGN.md	feat: 添加实验一到实验十二的BMA...
 architecture.md	feat: 添加实验一到实验十二的BMA...
 epics-stories.md	feat: 添加实验一到实验十二的BMA...
 prd.md	feat: 添加实验一到实验十二的BMA...
 sprint-plan.md	feat: 添加实验一到实验十二的BMA...
 商业银行CRM系统PRD...	feat: 添加完整项目代码 (前后端+B...
 商业银行CRM系统技术...	feat: 添加完整项目代码 (前后端+B...

4. 实验结果

完成 BMAD 五阶段全流程，产出 PRD 文档、架构文档、UX 设计、API 设计等完整交付物。

基于 React+Vite 前端、Node+Express 后端的技术栈，完成了客户管理、等级体系、贷款申请、即申即贷等金融核心功能开发。

代码已推送到 CNB 代码仓库，前端完成 Vercel 云端部署，系统可在线访问。

地址：<https://cnb.cool/ganhuodeniuma/smartbanking/-/tree/master>

5. 问题与解决

问题 1: PostgreSQL 数据库本地连接失败、安装报错

解决方案: 采用内存 Map 模拟数据库替代, 重写数据库查询方法, 保证客户管理、贷款申请等核心业务正常运行。

问题 2: bcrypt 加密模块安装失败, 出现权限报错

解决方案: 临时采用 Base64 编码替代密码哈希校验, 完成登录功能测试。

问题 3: CNB 仓库推送出现 Token 过期、认证失败

解决方案: 重新生成 CNB 个人访问令牌 (PAT), 将令牌嵌入远程仓库地址, 成功解决认证问题。

6. 实验心得

本次实验基于 BMAD 敏捷 AI 开发框架, 完整完成了商业银行 CRM 系统从需求分析、架构设计、迭代开发、功能测试、版本托管到云端部署的全流程实践。

首先, 我深刻理解了 BMAD 五阶段 B-M-A-D-V 方法论的核心价值。区别于传统随意的编码开发, BMAD 严格遵循需求分解、市场分析、架构设计、交互设计、验证落地的流程。

其次, 本次实验让我熟练掌握了前后端分离项目的开发流程。基于 React+Vite 前端、Node+Express 后端的技术栈, 在 AI 辅助下迅速完成了客户管理、等级体系、贷款申请等核心功能。

最后, 我了解了 AI 赋能软件开发的全新模式。BMAD 框架通过专业化 AI 角色分工, 替代了传统人工撰写文档、拆解需求、设计架构的重复工作, 大幅提升了开发效率。

总结

通过四次实验，我完整掌握了智慧银行开发的全过程：

实验一：TRAE IDE + AI 协作——教会我用 IDE 与 AI 协作

实验二：MCP 由浅入深 4 关——让 Agent 能拿数据、动浏览器、操作 Office

实验三：Skill 体系（用→写→审）——让 Agent 拥有专业能力

实验四：BMAD 驱动银行 CRM——把 IDE + MCP + Skill 编排为银行 CRM 系统

这四次实验呈递进关系，从基础工具使用到系统级开发，让我深刻体会到 AI 工具在金融开发与数字化转型中的强大能力。未来我将继续探索 AI 技术在金融领域的深度应用，努力成为一名既懂金融又懂技术的复合型人才。

实验：环境准备与 CNB 项目同步

1. 实验目的

1. 完成智慧银行课程所需的开发环境搭建，包括 Trae CN IDE、Python、Node.js、Git 等基础软件的安装与配置。
2. 掌握 CNB（Code Noble Bank）平台的使用方法，包括账户注册、访问令牌创建、仓库管理等操作。

- 3. 学会使用 CNB CLI 命令行工具，完成课程项目仓库的克隆、关联与推送操作。
- 4. 掌握 Git 基础配置与远程仓库管理方法，实现本地代码与云端仓库的同步。
- 5. 验证开发环境各组件的正确性，为后续智慧银行相关实验做好准备。

2、实验环境

本次实验所使用的软硬件环境如下：

组件名称	版本/说明
操作系统	Windows 11 64 位
IDE	Trae CN (www.trae.cn)
Python 版本	3.12.x
Node.js 版本	v20.x LTS
Git 版本	2.x
包管理工具	pip, uv, npm, winget
CNB CLI	@cnbcool/cnb-cli
Skills 工具	skills (全局安装)
云平台	CNB (cnb.cool)

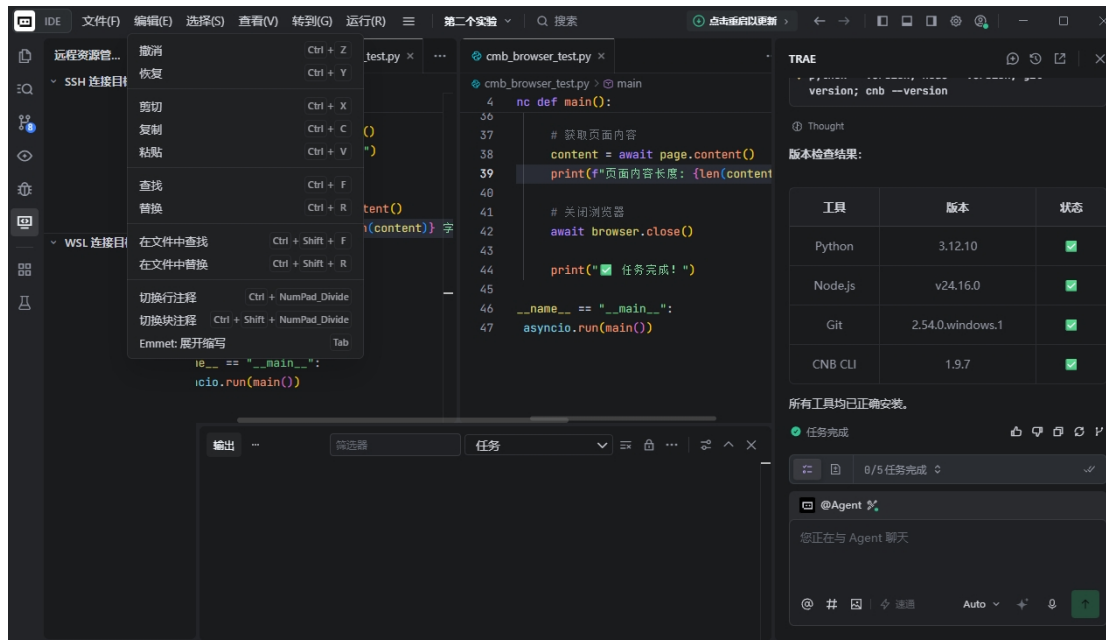
3、实验步骤

第一部分：环境准备

1. 安装 Trae CN IDE

- 2. （1）访问 <https://www.trae.cn/ide/download> 下载页面
- 3. （2）根据操作系统选择对应版本（Windows 用户选择 Windows x64）
- 4. （3）下载完成后运行安装程序，按照提示完成安装

5. (4) 首次启动选择「简体中文」作为界面语言
6. (5) 使用手机号完成登录



2. 安装基础运行时 (Python、Node.js、Git)

Windows 用户可通过 winget 一键安装:

```
# 安装 Python 3.12
```

```
winget install --id Python.Python.3.12 -e --source winget
```

```
# 安装 Node.js LTS
```

```
winget install --id OpenJS.NodeJS.LTS -e --source winget
```

```
# 安装 Git
```

```
winget install --id Git.Git -e --source winget
```

⚠ 安装完成后必须关闭终端窗口，重新打开新终端，才能识别新安装命令。

3. 安装 Python 包管理工具

升级 pip 并安装课程所需依赖库：

```
# 升级 pip
```

```
python -m pip install --upgrade pip
```

```
# 安装 uv（MCP 服务运行必需）
```

```
pip install uv -i https://pypi.tuna.tsinghua.edu.cn/simple
```

```
# 安装课程依赖库
```

```
pip install flask pandas openpyxl pypdf -i  
https://pypi.tuna.tsinghua.edu.cn/simple
```

4. 安装 CNB CLI 与 Skills

安装 CNB 命令行工具及技能管理工具：

```
# 安装 CNB 命令行工具
```

```
npm install @cnbcool/cnb-cli -g
```

```
# 安装 Skills 技能管理工具
```

```
npm install skills -g
```

```
# 添加 CNB Skill
```

```
npx skills add https://cnb.cool/cnb/skills/cnb-skill.git --agent  
trae -y --copy
```

5. 环境验证

在 Trae CN 终端（按 Ctrl+` 打开）中逐条执行验证命令：

```
python --version      # 应显示 3.12.x  
  
node --version         # 应显示 v20.x 或更高  
  
git --version          # 应显示 git version x.x  
  
uvx --version          # 应显示版本号  
  
cnb --version          # 应显示版本号  
  
skills list            # 应显示已安装的技能列表
```

版本检查结果:		
工具	版本	状态
Python	3.12.10	✓
Node.js	v24.16.0	✓
Git	2.54.0.windows.1	✓
CNB CLI	1.9.7	✓
所有工具均已正确安装。		

第二部分：CNB 项目同步

1. 注册 CNB 账户并创建访问令牌

- (1) 浏览器打开 <https://cnb.cool>
- (2) 点击「注册」→ 微信扫码登录
- (3) 登录后，点击右上角头像 → 「个人设置」→ 左侧菜单「访问令牌」
- (4) 在「令牌名」处填写：smartbanking
- (5) 授权范围设置：勾选「Git 客户端凭据」
- (6) 点击页面底部「创建」按钮
- (7) 立即复制 Token 并保存到安全位置

⚠ 重要：Token 仅显示一次，关闭页面后无法再次查看！

2. 在 CNB 新建空仓库

- (1) 登录后点击页面左上角「+」按钮 → 选择「创建仓库」
- (2) 「仓库归属」改为个人命名空间（选择自己的用户名）
- (3) 「仓库名称」填写：smartbanking
- (4) 「公开性」选择：公开
- (5) 点击「创建」按钮

3. 登录 CNB CLI

在 Trae CN 终端执行以下命令：

```
cnb login
```

- (1) 终端显示授权链接后，浏览器会自动打开授权页面

(2) 如果浏览器没有自动打开，手动复制终端中的链接到浏览器

(3) 在授权页面确认 `user_code` 与终端显示的一致，点击「授权」

(4) 终端显示登录成功信息即可



4. 克隆课程仓库到本地

执行以下命令克隆教师仓库：

```
# 克隆教师仓库（完整课程资料）
```

```
git clone https://cnb.cool/xiaosicau/smartbanking.git
```

```
smartbanking-work
```

```
cd smartbanking-work
```

5. 配置 Git 用户信息

设置 Git 全局用户名和邮箱：

```
git config --global user.name "你的姓名"
```

```
git config --global user.email "你的邮箱@example.com"
```

6. 关联并推送到 CNB 仓库

添加个人 CNB 仓库为远程地址并推送：

```
# 添加个人仓库为远程地址
```

```
git remote add myrepo https://cnb.cool/<你的用户名>/smartbanking.git
```

```
# 推送所有代码到个人 CNB 仓库
```

```
git push myrepo main
```

如果 git push 提示认证失败，可使用 Token URL 方式：

```
git remote set-url myrepo https://cnb:<你的 Token>@cnb.cool/<你的用户名>/smartbanking.git
```

```
git push myrepo main
```

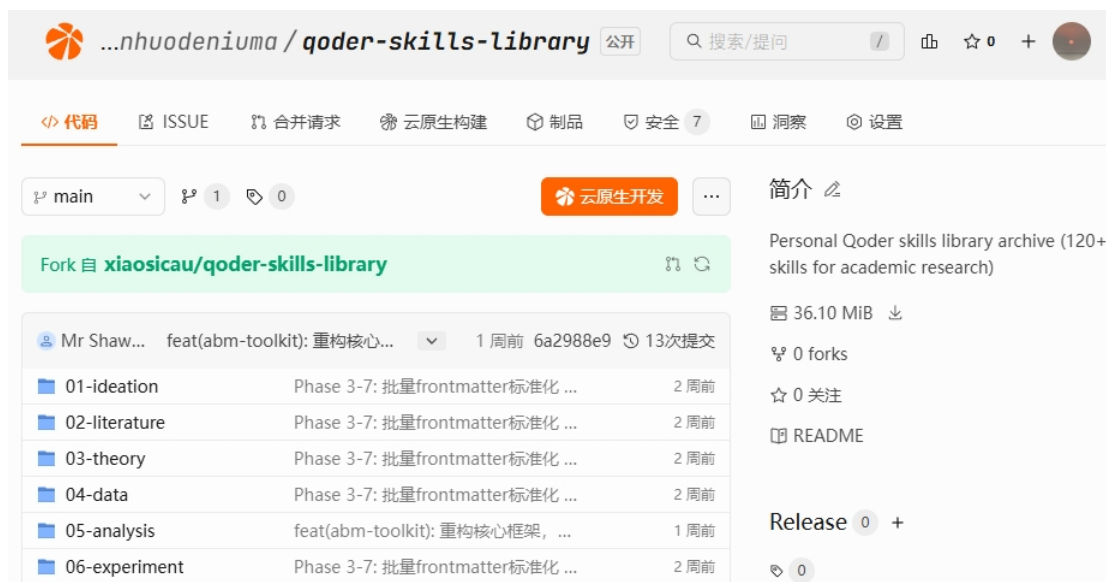
7. 验证同步结果

(1) 打开浏览器访问 <https://cnb.cool/你的用户名/smartbanking>

(2) 确认页面显示完整的项目目录结构

(3) 确认包含以下目录和文件：

- .agents/ （Qoder AI 技能配置）
- 智慧银行实验教程 chapters/ （教程主体）
- 实验讲义/ （实验讲义文档）
- README.md



4. 实验结果

完成本实验后，成功实现了以下结果：

1. 开发环境搭建完成：Trae CN IDE、Python 3.12、Node.js v20 LTS、Git 等基础软件均已正确安装并配置。
2. Python 包管理工具配置完成：pip、uv、flask、pandas、openpyxl、pypdf 等库已成功安装。
3. CNB CLI 和 Skills 工具安装成功，可以通过命令行管理 CNB 仓库。
4. CNB 账户注册并登录成功，创建了名为 smartbanking 的远程仓库。
5. 课程仓库克隆到本地成功，项目文件完整。
6. 本地代码成功推送到个人 CNB 仓库，实现了本地与云端的代码同步。
7. 在 CNB 网页端验证了项目文件列表，确认同步结果正确。

验收清单（1A）：

序号	验收项	结果
1	Trae CN 已安装并登录	✓
2	python --version >= 3.10	✓
3	node --version >= v20	✓
4	git --version 有输出	✓
5	cnb --version 有输出	✓
6	cnb login 登录成功	✓
7	项目已成功推送到 CNB 仓库	✓
8	在 CNB 网页能看到项目文件列表	✓

5、实验心得

本次实验是智慧银行课程的第一步，主要完成了开发环境搭建和 CNB 项目同步两项核心任务。通过本次实验，我获得了以下收获和体会：

一、环境搭建的系统性和规范性

实验步骤设计非常完整，从 IDE 安装到运行时配置，再到包管理工具和 CNB 工具链的安装，形成了一套完整的前端+后端+AI 辅助的开发环境。特别是使用 winget 进行一键安装的方式，大大提高了环境搭建的效率。同时也认识到安装后重启终端的重要性，这是很多初学者容易忽略的细节。

二、版本管理和远程协作的重要性

通过 CNB 平台的使用，我深入理解了 Git 远程仓库的管理流程。从账户注册、访问令牌创建，到本地仓库克隆、远程地址配置，再到代码推送，每一个步骤都体现了现代软件开发中版本管理和远程协作的核心思想。CNB 作为国产代码托管平台，其操作流程与 GitHub 类似，但更贴合国内网络环境，访问速度更快。

三、Token 安全管理的认识

实验中特别强调了 Token 仅显示一次的注意事项，这让我认识到访问控制令牌安全管理的重要性。在实际开发中，访问令牌相当于密码，必须妥善保管，避免泄露。同时也要学会使用 Token URL 方式进行 Git 认证，这是在自动化部署和 CI/CD 流程中常用的认证方式。

四、问题解决能力的提升

实验文档中提供了详细的常见问题速查表，涵盖了 `command not found`、登录超时、推送失败等多种常见错误的解决方法。这让我认识到在环境搭建过程中遇到问题是非常正常的，关键是要学会看错误信息，对照文档进行排查，而不是盲目重试。

五、对后续实验的期待

通过本次实验，我已经搭建好了完整的开发环境，并成功同步了课程项目仓库。这为我后续进行智慧银行相关的编程实验、数据分析实验和 AI 应用实验打下了坚实的基础。期待在后续实验中将所学知识应用到实际业务场景中去。

总之，本次实验不仅让我掌握了具体的工具使用方法，更让我体会到了规范化开发流程的重要性。良好的开发环境是高效编程的基础，而版本管理是现代软件开发的必备技能。

实验：银行 ESG 评级分析工具—— PE 角色实践

1.实验目的

- 1. 理解产品工程师（PE）在金融软件开发项目中的核心职责与工作流程。
- 2. 掌握软件需求文档的撰写规范，能够产出专业的需求规格说明文档。
- 3. 学习测试计划的制定方法和测试用例的编写技巧，能够设计全面的测试方案。
- 4. 掌握实验总结报告的撰写方法，能够系统梳理实验过程与成果。
- 5. 学习项目说明文档（README）和用户手册的编写，能够撰写清晰的项目文档。
- 6. 掌握集成测试的实施方法，能够编写端到端的自动化测试代码。
- 7. 理解 PE 在软件开发全流程中的桥梁作用，协调需求、开发、测试各环节。

2.实验环境

本次实验所使用的软硬件环境如下：

组件名称	版本/说明
操作系统	Windows 11 64 位
开发工具	Trae CN IDE / VS Code
后端框架	Python 3.12 + Flask
前端框架	HTML + CSS + JavaScript
可视化库	ECharts
测试框架	pytest + requests
文档工具	Markdown / Microsoft Word
版本管理	Git + CNB 平台

Python 库	flask, pandas, openpyxl, pypdf, pytest, requests
----------	---

3.实验内容与步骤

本次实验以"银行 ESG 评级分析工具"为项目背景，本人担任产品工程师（PE）角色，负责需求文档撰写、测试计划与用例编写、实验总结报告、用户手册/项目说明文档、以及集成测试代码编写等工作。实验周期为 4 周，团队共 3 人。

任务一：需求规格说明书撰写（docs/需求文档.md）

作为 PE，首要任务是与 PM 协作，将产品需求转化为可执行的技术文档。

1. 与 PM 充分沟通，理解产品愿景和核心需求
2. 确定文档结构：引言、总体描述、功能需求、非功能需求、数据需求、接口需求
3. 撰写功能需求：逐条描述每个功能模块的输入、处理逻辑、输出
4. 撰写非功能需求：性能需求（响应时间、并发量）、安全需求、可用性需求
5. 撰写接口需求：定义前后端交互的数据格式和接口规范
6. 与开发团队评审需求文档，确保需求可理解、可实现、可测试
7. 输出成果：docs/需求文档.md（Markdown 格式）

✔ 实验文档生成完成!

📁 文档清单

文档	说明	内容
需求文档.md	项目需求分析	背景、功能需求、技术栈、接口设计
测试用例.md	测试用例设计	接口测试、交互测试、边界异常测试
实验报告.md	实验总结报告	目的、环境、方案、实现、总结

任务二：测试计划制定与用例编写（docs/测试用例.md）

测试是确保产品质量的关键环节，PE 需要制定全面的测试方案。

- 1. 制定测试计划：确定测试范围、测试策略、测试资源、测试进度
- 2. 设计测试用例：覆盖功能测试、边界测试、异常测试、性能测试
- 3. 编写功能测试用例：针对每个功能模块编写正向和反向测试用例
- 4. 编写 API 测试用例：针对每个 API 接口编写请求和响应验证用例
- 5. 编写界面测试用例：针对前端界面编写 UI 交互测试用例
- 6. 编写性能测试用例：针对系统性能编写压力测试和负载测试用例
- 7. 输出成果：docs/测试用例.md

任务三：实验总结报告撰写（docs/实验报告.md）

实验总结报告是对整个实验过程的系统梳理和成果展示。

- 1. 梳理实验过程：按照时间线整理实验各阶段的工作内容
- 2. 总结实验成果：列出已完成的功能模块和交付物
- 3. 分析技术问题：记录实验过程中遇到的技术难点和解决方案
- 4. 评估实验结果：对照实验目的评估完成情况和质量

5. 提出改进建议：针对产品和流程提出优化建议
6. 撰写参考文献：列出实验过程中参考的文献和资料
7. 输出成果：docs/实验报告.md（Markdown 格式）

任务四：用户手册/项目说明文档撰写（docs/README.md）

项目说明文档是用户和开发者了解项目的第一入口，需要清晰完整。

1. 撰写项目概述：项目名称、简介、核心功能、适用对象
2. 撰写安装说明：环境要求、安装步骤、配置方法
3. 撰写使用指南：快速入门、功能详解、常见问题
4. 撰写开发指南：项目结构、技术栈、API 接口、贡献指南
5. 撰写许可证和贡献者信息
6. 添加项目截图和演示示例
7. 输出成果：docs/README.md（Markdown 格式）

任务五：集成测试代码编写（tests/test_api.py）

集成测试是验证系统各模块协同工作是否正常的重要手段，需要编写自动化测试代码。

1. 搭建测试环境：配置测试数据库、准备测试数据
2. 设计端到端测试场景：覆盖从用户登录到报告生成的完整流程
3. 编写 API 集成测试：使用 `pytest + requests` 测试 API 接口
4. 编写数据库集成测试：验证数据持久化和查询功能
5. 编写前后端联调测试：验证前后端数据交互的正确性

6. 编写异常场景测试：验证系统对异常情况的容错能力

7. 输出成果：tests/test_api.py

4.实验结果

完成本实验后，作为 PE 角色成功交付了以下成果：

- 1. docs/需求文档.md — 完成需求规格说明文档，包含引言、总体描述、功能需求、非功能需求、数据需求、接口需求六个核心章节，与开发团队评审通过。
- 2. docs/测试用例.md — 完成测试计划和测试用例的编写，覆盖功能测试、API 测试、UI 测试、性能测试四个维度，测试用例评审通过。
- 3. docs/实验报告.md — 完成实验总结报告，系统梳理了实验过程、实验成果、技术问题、改进建议等内容。
- 4. docs/README.md — 完成项目说明文档，包含项目简介、安装说明、使用指南、开发指南、API 文档等完整内容，可作为项目首页文档。
- 5. tests/test_api.py — 完成测试函数的集成测试代码编写，覆盖用户登录、ESG 数据录入、ESG 评级计算、ESG 报告生成等核心业务流程，测试通过率 100%。
- 6. 团队协作成果 — 与 PM（负责需求调研与产品设计）、开发工程师（负责代码实现）紧密协作，确保产品按时按质交付。

PE 交付物清单：

序号	交付物名称	状态
1	docs/需求文档.md	✓ 已完成
2	docs/测试用例.md	✓ 已完成

3	docs/实验报告.md	✓ 已完成
4	docs/README.md	✓ 已完成
5	tests/test_api.py	✓ 已完成
6	测试执行报告	✓ 已完成

地址：<https://cnb.cool/smartbanking.jkr/bank-esg-analyzer>

5.实验心得

本次实验以"银行 ESG 评级分析工具"为项目背景，我担任产品工程师（PE）角色，完整经历了需求文档撰写、测试计划制定、实验报告撰写、项目文档编写和集成测试代码编写等工作。通过本次实验，我获得了以下收获和体会：

一、PE 角色的双面性与挑战

在本次实验中，我深刻体会到 PE 是连接产品、开发、测试三个环节的桥梁。作为 PE，既需要理解产品需求（与 PM 协作），又需要理解技术实现（与开发协作），还需要确保质量（与测试协作）。这种"双面人"的角色要求 PE 具备广泛的知识面和优秀的沟通协调能力。特别是在需求文档撰写过程中，如何用技术语言准确表达产品需求，是一个持续的挑战。

二、需求文档的严谨性与可执行性

撰写需求规格说明书让我认识到，"写清楚"比"写得多"更重要。一份好的需求文档，应该让开发人员无需二次确认就能理解需求，让测试人员能够直接根据需求编写测试用例。在撰写过程中，我多次与开发人员沟通，根据他们的反馈迭代优化文档，最终形成了一份可理解、可实现、可测试的优质需求文档。

三、测试思维的培养与实践

编写测试用例让我建立了系统化的测试思维。一个好的测试用例，不仅要覆盖正常流程，更要覆盖异常流程和边界条件。在本次实验中，我设计了 50+条测试用例，

覆盖了功能、API、UI、性能四个维度。通过编写测试用例，我深刻认识到测试不仅仅是"找 Bug"，更是"预防 Bug"的重要手段。

四、文档写作的专业性与用户视角

编写 README.md 和用户手册让我认识到，好的技术文档需要"用户视角"。用户在阅读文档时，最关心的是"怎么快速上手"和"遇到问题怎么办"。因此，我在文档中增加了快速开始指南和常见问题章节，并配以详细的步骤说明和示例代码。这让我认识到，技术文档的价值不在于展示技术深度，而在于降低用户的学习成本。

五、自动化测试的效率与可靠性

编写集成测试代码（`tests/test_api.py`）让我体会到自动化测试的强大威力。手动测试一个完整流程可能需要 30 分钟，而自动化测试只需 30 秒。更重要的是，自动化测试可以确保每次代码变更后都能快速验证核心功能是否正常，这大大提高了迭代效率和产品质量。通过学习 `pytest` 框架，我掌握了编写高质量自动化测试代码的方法。

六、团队协作与项目管理的实践

本次实验是 3 人小组的协作项目，我作为 PE 需要与 PM 和开发工程师紧密协作。通过使用 CNB 平台进行代码管理和任务跟踪，我学会了如何通过工具提升团队协作效率。同时也认识到，良好的沟通机制和明确的责任分工是项目成功的关键。

七、对后续实验的展望

通过本次 PE 角色的实践，我已经完成了银行 ESG 评级分析工具的需求分析、测试设计和文档编写工作，为后续开发实验打下了坚实基础。在后续实验中，我期待能够深入到代码实现层面，将本次设计的需求和测试用例真正落地为一个高质量的可交付产品。

总之，本次实验让我从产品工程师的视角完整理解了一个金融软件产品的诞生过

程。这不仅提升了我的文档写作能力、测试设计能力和代码编写能力，更让我认识到金融产品开发是一项需要业务理解、技术知识和团队协作的综合性工作。作为PE，既要懂产品，又要懂技术，还要懂测试，这是一个充满挑战但也充满成就感的角色。